Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ЛАБОРАТОРНАЯ 1

По дисциплине Контейнеризация и оркестрация приложений

Тема работы Знакомство с экосистемой Docker: от базовых команд к диагностике сервиса

Обучающийся Ямалеева Азалия Фиразовна

Факультет Прикладной информатики

Группа К3320

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Ямалеева.А.Ф
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 ВЕРИФИКАЦИЯ ОКРУЖЕНИЯ И АРХИТЕКТУРЫ 4
1.1 Проверка Docker 4
1.2 Ответы на вопросы 4
2 ПЕРВЫЕ ШАГИ “ИГРУШЕЧНЫЕ” ОБРАЗЫ 6
2.1 Изучение простых образов и статусов контейнеров 6
2.2 Ответы на вопросы 7
3 ЖИЗНЕННЫЙ ЦИКЛ КОНТЕЙНЕРА 9
3.1 Изучение жизненного цикла контейнера 9
3.2 Ответы на вопросы 10
4.1 Запуск контейнера с ошибками 10
ЗАКЛЮЧЕНИЕ 13

ВВЕДЕНИЕ

Цель данной работы заключается в изучении базовых команд утилиты Docker.

Персональные данные:

Параметр	Значение
Номер ИСУ	468183
Персональный порт	5513
Персональный префикс	yamaleeva

1 ВЕРИФИКАЦИЯ ОКРУЖЕНИЯ И АРХИТЕКТУРЫ

1.1 Проверка Docker

Из вывода терминала была получена информация. Версии клиента и сервера совпадают и равны 28.5.1, демон работает под управлением Linux на архитектуре x86_64 (Docker Desktop использует виртуальную машину WSL 2). До начала работы в локальном хранилище находилось 2 образа общим размером 3,836 ГБ, запущенных контейнеров не было.

```
yamaleeva@ALOVINNE> docker version --format "Client: {{.Client.Version}}, Server: {{.Server.Version}}"
Client: 28.5.1, Server: 28.5.1
yamaleeva@ALOVINNE> docker info --format "{{.OSType}}/{{.Architecture}}"
linux/x86_64
```

Рисунок 1 – Версии клиента и сервера Docker, тип ОС и архитектура демона

```
yamaleeva@ALOVINNE> docker system df --format "table {{.Type}}\t{{.TotalCount}}\t{{.Active}}\t{{.Size}}"
TYPE            TOTAL     ACTIVE    SIZE
Images          2         0         3.836GB
Containers      0         0         0B
Local Volumes   0         0         0B
Build Cache     12        0         108.4MB
```

Рисунок 2 – Использование дискового пространства объектами Docker

```
yamaleeva@ALOVINNE> echo "Student: $(whoami)@$(hostname)"; echo "Date: $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')"; docke
r version --format "Docker {{.Server.Version}} on {{.Server.Os}}/{{.Server.Architecture}}"
Student: alovinne\yujau@alovinne
Date: 2026-10-05 21:47:19
Docker 28.5.1 on linux/
template: version:1:54: executing "version" at <.Server.Architecture>: can't evaluate field Architecture in type *types.
Version
```

Рисунок 3 – Артефакт окружения

1.2 Ответы на вопросы

1. Что произойдёт, если запустить docker system prune без флагов? Какие данные будут потеряны?

Команда docker system prune удаляет неиспользуемые объекты Docker: все остановленные контейнеры, неиспользуемые сети, образы без тегов и неиспользуемый кэш сборки. Образы, у которых есть теги, и тома без дополнительных флагов (-a и --volumes) не удаляются. Будут

потеряны данные, которые хранились в записываемом слое остановленных контейнеров, а также кэш сборки, из-за чего последующие сборки образов станут дольше.

2. Почему команда docker version может завершиться ошибкой, даже если Docker установлен?

Команда docker version обращается к демону через клиент, поэтому ошибка возникает, когда связь с демоном невозможна. Основные причины: демон не запущен (например, не запущен Docker Desktop), у пользователя нет прав на доступ к сокету /var/run/docker.sock (пользователь не состоит в группе docker), а также неверно заданы переменная DOCKER_HOST или контекст Docker. В таком случае выводится только версия клиента.

2 ПЕРВЫЕ ШАГИ “ИГРУШЕЧНЫЕ” ОБРАЗЫ

2.1 Изучение простых образов и статусов контейнеров

Образ hello-world загружается из реестра Docker Hub, полное имя образа docker.io/library/hello-world:latest. Внутри контейнера выполняется исполняемый файл /hello, который выводит приветственное сообщение и завершает работу. Значение поля PRETTY_NAME в образе alpine:3.18 равно «Alpine Linux v3.18», размер образа составляет 11,5 МБ. Два контейнера на основе alpine:3.18 были запущены в фоновом режиме, оба имеют статус Up.

```
yamaleeva@ALOVINNE> docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Рисунок 4 – Запуск образа hello-world

```
yamaleeva@ALOVINNE> docker run --rm alpine:3.18 cat /etc/os-release
Unable to find image 'alpine:3.18' locally
3.18: Pulling from library/alpine
44cf07d57ee4: Pull complete
Digest: sha256:de0eb0b3f2a47ba1eb89389859a9bd88b28e82f5826b6969ad604979713c2d4f
Status: Downloaded newer image for alpine:3.18
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.18.12
PRETTY_NAME="Alpine Linux v3.18"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
```

Рисунок 5 – Содержимое файла /etc/os-release в образе alpine:3.18

```
yamaleeva@ALOVINNE> docker images alpine:3.18 --format "{{.Size}}"
11.5MB
```

Рисунок 6 – Размер образа alpine:3.18

```
yamaleeva@ALOVINNE> docker run -d --name yamaleeva-alpine-1 alpine:3.18 sleep 300
c4a28049313136c335b5c76841adf0ed44b0ec6b7dcdb7f57acf11855c20ef04
yamaleeva@ALOVINNE> docker run -d --name yamaleeva-alpine-2 alpine:3.18 sh -c "echo 'Hello from $(hostname)' && sleep 30
0"
eba20c639d3f4247abcf4992281a7282b89ffbbab96244938f2c55642b05ec9f
yamaleeva@ALOVINNE> docker ps --filter "name=yamaleeva-alpine"
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS          PORTS     NAMES
eba20c639d3f   alpine:3.18   "sh -c 'echo 'Hello …"   8 seconds ago    Up 7 seconds              yamaleeva-alpine-2
c4a280493131   alpine:3.18   "sleep 300"              24 seconds ago   Up 24 seconds             yamaleeva-alpine-1
```

Рисунок 7 – Состояние контейнеров yamaleeva-alpine-1 и yamaleeva-alpine-2

2.2 Ответы на вопросы

1. Почему образ hello-world завершается сразу после запуска, а alpine:3.18 sleep 300 работает 5 минут?

Контейнер работает, пока работает его главный процесс. /hello выводит текст и завершается.В случае alpine:3.18 sleep 300 главным процессом является sleep, который ожидает 300 секунд, и всё это время контейнер находится в состоянии Up.

2. Что произойдёт, если убрать флаг --rm из команды docker run --rm hello-world? Как удалить «забытые» контейнеры?

Контейнер останется в состоянии Exited и будет виден в docker ps -a. Удалить можно командой docker rm с именем контейнера или docker container prune

3 ЖИЗНЕННЫЙ ЦИКЛ КОНТЕЙНЕРА

3.1 Изучение жизненного цикла контейнера

Для изучения жизненного цикла использовался образ nginx:alpine. Контейнер последовательно был создан, запущен, остановлен, перезапущен, принудительно завершён и удалён. После каждого шага проверялся его статус.

```
yamaleeva@ALOVINNE> docker create --name yamaleeva-nginx nginx:alpine
dcec95a5ef20d5ef8a553b521d9d8e994f222fcb68a359d28f235850d10ab0f6
yamaleeva@ALOVINNE> docker ps -a --filter "name=yamaleeva-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}
"
NAMES             STATUS    CREATED AT
yamaleeva-nginx   Created   2026-10-05 22:15:55 +0300 MSK
yamaleeva@ALOVINNE> docker start yamaleeva-nginx
yamaleeva-nginx
yamaleeva@ALOVINNE> docker ps -a --filter "name=yamaleeva-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}
"
NAMES             STATUS         CREATED AT
yamaleeva-nginx   Up 5 seconds   2026-10-05 22:15:55 +0300 MSK
yamaleeva@ALOVINNE> docker stop yamaleeva-nginx
yamaleeva-nginx
yamaleeva@ALOVINNE> docker ps -a --filter "name=yamaleeva-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}
"
NAMES             STATUS                     CREATED AT
yamaleeva-nginx   Exited (0) 4 seconds ago   2026-10-05 22:15:55 +0300 MSK
yamaleeva@ALOVINNE> docker restart yamaleeva-nginx
yamaleeva-nginx
yamaleeva@ALOVINNE> docker ps -a --filter "name=yamaleeva-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}
"
NAMES             STATUS         CREATED AT
yamaleeva-nginx   Up 5 seconds   2026-10-05 22:15:55 +0300 MSK
yamaleeva@ALOVINNE> docker kill yamaleeva-nginx
yamaleeva-nginx
yamaleeva@ALOVINNE> docker ps -a --filter "name=yamaleeva-nginx" --format "table {{.Names}}\t{{.Status}}\t{{.CreatedAt}}
"
NAMES             STATUS                       CREATED AT
yamaleeva-nginx   Exited (137) 5 seconds ago   2026-10-05 22:15:55 +0300 MSK
yamaleeva@ALOVINNE> docker rm yamaleeva-nginx
yamaleeva-nginx
yamaleeva@ALOVINNE>
```

Рисунок 8 – Последовательность команд управления контейнером yamaleeva-nginx

Таблица 1 – Состояния контейнера yamaleeva-nginx

Шаг	Статус
create	Created
start	Up 5 seconds
stop	Exited (0) 4 seconds ago
restart	Up 5 seconds
kill	Exited (137) 5 seconds ago
rm	контейнер удален

После команды docker stop контейнер завершился с кодом 0, то есть остановка прошла штатно. После команды docker kill код завершения равен 137 (128 + 9), что соответствует сигналу SIGKILL.

3.2 Ответы на вопросы

1. Какой сигнал отправляет docker stop по умолчанию?

По умолчанию docker stop отправляет главному процессу контейнера сигнал SIGTERM. Если процесс не завершился в течение 10 секунд, Docker отправляет SIGKILL.

2. Почему docker kill может привести к потере данных в контейнере с БД?

SIGKILL убивает процесс сразу. БД не успевает сбросить данные на диск и закрыть транзакции, поэтому последние изменения могут пропасть.

3. Что произойдёт, если выполнить docker rm для запущенного контейнера? Какой флаг нужен для принудительного удаления?

Команда docker rm для запущенного контейнера завершится ошибкой, так как Docker не удаляет работающие контейнеры. Для принудительного удаления используется флаг -f (--force).

4 ДИАГНОСТИКА РЕАЛЬНОГО СЕРВИСА

4.1 Запуск контейнера с ошибками

Контейнер yamaleeva-pg-broken был запущен без переменной окружения POSTGRES_PASSWORD. Он завершился со статусом Exited (1). В журнале содержится сообщение «Database is uninitialized and superuser password is not specified», то есть база данных не инициализирована, а пароль

суперпользователя не задан. Код завершения, полученный командой docker inspect, равен 1.

```
yamaleeva@ALOVINNE> docker run -d --name yamaleeva-pg-broken -p 5513:5432 postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
c95afe7c234f: Pull complete
eb4d92aaee0a: Pull complete
75a4dff090e8: Pull complete
0d39881c83f7: Pull complete
0d791b2b75ae: Pull complete
afe9a40fb214: Pull complete
b65147f15943: Pull complete
ca5e97c0cd7d: Pull complete
170d9ea1cd85: Pull complete
bc176b99a33a: Pull complete
7b9c6b8e5e6e: Pull complete
08a5cd487deb: Pull complete
6b37362b3da7: Pull complete
53fa82bdba52: Pull complete
Digest: sha256:724292da1f2e50bdccfc3302ce75bbba7f4a6076701b588cc795fcac65683550
Status: Downloaded newer image for postgres:15
3bc4ac8f03cf41bad9cb624e185c268b0404fcf70c11baa159680f4ff2520af7
```

Рисунок 9 – Запуск контейнера yamaleeva-pg-broken без переменной POSTGRES_PASSWORD

```
yamaleeva@ALOVINNE> docker ps -a --filter "name=yamaleeva-pg"
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS                      PORTS     NAMES
3bc4ac8f03cf   postgres:15   "docker-entrypoint.s…"   48 seconds ago   Exited (1) 44 seconds ago             yamaleeva-p
g-broken
```

Рисунок 10 – Статус контейнера yamaleeva-pg-broken

```
yamaleeva@ALOVINNE> cmd /c "docker logs --timestamps yamaleeva-pg-broken 2>&1" | Select-Object -Last 20
2026-10-05T19:33:48.613559307Z Error: Database is uninitialized and superuser password is not specified.
2026-10-05T19:33:48.613839260Z        You must specify POSTGRES_PASSWORD to a non-empty value for the
2026-10-05T19:33:48.613850783Z        superuser. For example, "-e POSTGRES_PASSWORD=password" on "docker run".
2026-10-05T19:33:48.613858679Z
2026-10-05T19:33:48.613864781Z        You may also use "POSTGRES_HOST_AUTH_METHOD=trust" to allow all
2026-10-05T19:33:48.613871194Z        connections without a password. This is *not* recommended.
2026-10-05T19:33:48.613877286Z
2026-10-05T19:33:48.613958800Z        See PostgreSQL documentation about "trust":
2026-10-05T19:33:48.613967608Z        https://www.postgresql.org/docs/current/auth-trust.html
yamaleeva@ALOVINNE> docker inspect yamaleeva-pg-broken --format='{{.State.ExitCode}}'
1
```

Рисунок 11 – Журнал и код завершения контейнера yamaleeva-pg-broken

```
yamaleeva@ALOVINNE> docker run -d --name yamaleeva-pg-fixed -e POSTGRES_PASSWORD=Pass_5513 -e POSTGRES_USER=user_468183
-p 5513:5432 postgres:15
9d8be5b42b56ecc433a287a2b009dfd24f04598f886ae14418405b41499e68f5
yamaleeva@ALOVINNE> Start-Sleep 15
yamaleeva@ALOVINNE> docker exec yamaleeva-pg-fixed psql -U user_468183 -c "SELECT version();"
                                                 version
---------------------------------------------------------------------------------------------------------
 PostgreSQL 15.19 (Debian 15.19-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)
```

Рисунок 12 – Запуск контейнера yamaleeva-pg-fixed и проверка запроса к базе данных



Рисунок 13 – Артефакт выполнения работы

Причина ошибки заключается в том, что официальный образ postgres по умолчанию требует пароль суперпользователя и без него не запускает инициализацию базы данных. Это сделано из соображений безопасности. Для исправления контейнер yamaleeva-pg-fixed был запущен с переменными окружения POSTGRES_PASSWORD=Pass_5513 и POSTGRES_USER=user_468183, а порт 5513 хоста был проброшен на порт 5432 контейнера. После инициализации выполнен запрос SELECT version();, который вернул версию PostgreSQL 15.19. Статус контейнера Up и проброс порта 0.0.0.0:5513->5432/tcp подтверждены на рисунке 13.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы были изучены базовые команды Docker. Проверена установка и работоспособность Docker, получены сведения о версиях клиента и сервера и о состоянии локального хранилища. Запущены контейнеры из образов hello-world и alpine:3.18 и рассмотрено, как поведение контейнера зависит от его главного процесса. Изучен жизненный цикл контейнера.